AWS VPC Peering Project — EWB Cloud Labs

> **Project:** Establish private connectivity between two Amazon VPCs using VPC Peering
> **Cloud provider:** Amazon Web Services (AWS)
> **Region:** Asia Pacific (Mumbai) — `ap-south-1`
> **Operating system:** Amazon Linux 2023
> **Project type:** Hands-on cloud networking lab

Table of Contents

- Project Overview
- Objectives
- Architecture
- Resource Inventory
- Network and Routing Configuration
- Security Configuration
- Implementation Summary
- Validation and Test Results
- Screenshots and Evidence
- Key Learnings
- Cleanup

Project Overview

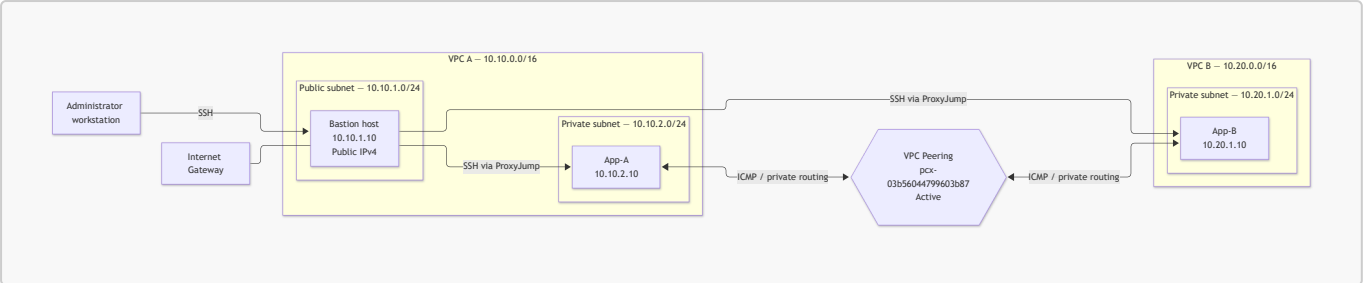
This project demonstrates how to connect two isolated Amazon Virtual Private Clouds (VPCs) using **AWS VPC Peering**. The environment contains a public subnet and a private subnet in VPC A, a private subnet in VPC B, a bastion host, and two private EC2 application instances.

A VPC peering connection provides private network routing between the VPC CIDR ranges. After configuring the peering connection and the required routes, connectivity was tested from App-A to App-B and from App-B to App-A. SSH access to App-B was also verified through the bastion host. A controlled security-group change was performed to demonstrate a connectivity failure and recovery.

Objectives

- Create two VPCs with non-overlapping CIDR blocks.
- Create and associate public and private subnets with their intended route tables.
- Configure an Internet Gateway for the public subnet in VPC A.
- Launch a bastion host and two private EC2 instances.
- Configure security groups for SSH and ICMP testing.
- Establish and activate a VPC peering connection.
- Add routes in both VPCs so private workloads can communicate.
- Validate baseline failure, bidirectional connectivity, SSH access, and controlled failure/recovery.

Architecture



Important: The diagram represents the completed lab configuration. The private application instances do not have public IP addresses. The bastion host is the SSH entry point for the private instances.

Resource Inventory

Resource	Name	Configuration
VPC A	`ewb-vpc-a`	`10.10.0.0/16`
VPC B	`ewb-vpc-b`	`10.20.0.0/16`
Public subnet in VPC A	`ewb-a-public`	`10.10.1.0/24`
Private subnet in VPC A	`ewb-a-private`	`10.10.2.0/24`
Private subnet in VPC B	`ewb-b-private`	`10.20.1.0/24`
Internet Gateway	`ewb-igw-a`	Attached to VPC A
Public route table	`ewb-rt-a-public`	Associated with `ewb-a-public`
Private route table A	`ewb-rt-a-private`	Associated with `ewb-a-private`
Private route table B	`ewb-rt-b-private`	Associated with `ewb-b-private`
Bastion EC2	`ewb-bastion`	`10.10.1.10`; public IPv4 enabled
App-A EC2	`ewb-app-a`	`10.10.2.10`; no public IP
App-B EC2	`ewb-app-b`	`10.20.1.10`; no public IP
VPC peering connection	`ewb-vpc-a-b-peer`	`pcx-03b56044799603b87`; Active
EC2 key pair	`ewb-lab`	Used for SSH authentication

Network and Routing Configuration

Route table entries

Route table	Destination	Target	Purpose
`ewb-rt-a-public`	`10.10.0.0/16`	`local`	Communication inside VPC A
`ewb-rt-a-public`	`0.0.0.0/0`	`ewb-igw-a`	Internet route for the public subnet

Route table	Destination	Target	Purpose
`ewb-rt-a-public`	`10.20.0.0/16`	VPC peering connection	Routes traffic from the bastion subnet toward VPC B
`ewb-rt-a-private`	`10.10.0.0/16`	`local`	Communication inside VPC A
`ewb-rt-a-private`	`10.20.0.0/16`	VPC peering connection	Routes App-A traffic toward VPC B
`ewb-rt-b-private`	`10.20.0.0/16`	`local`	Communication inside VPC B
`ewb-rt-b-private`	`10.10.0.0/16`	VPC peering connection	Return route from VPC B toward VPC A

The peering connection was created between VPC A (`10.10.0.0/16`) and VPC B (`10.20.0.0/16`) and reached **Active** status. Routes were added to the route tables used by the relevant subnets. The public route table's peering route is needed for the bastion host to reach App-B over the private network.

Security Configuration

Three security groups were configured for the lab:

- **`ewb-sg-bastion`** — inbound SSH (TCP 22) restricted to the administrator's public IP address with a `/32` mask.
- **`ewb-sg-app-a`** — inbound SSH (TCP 22) from the bastion host's private IP (`10.10.1.10/32`) and ICMP echo requests from App-B (`10.20.1.10/32`).
- **`ewb-sg-app-b`** — inbound SSH (TCP 22) from the bastion host's private IP (`10.10.1.10/32`) and ICMP echo requests from App-A (`10.10.2.10/32`).

The instances use Amazon Linux 2023. The application instances are private and are accessed through the bastion host. The lab uses the default network ACL configuration. Review the actual security-group and network ACL settings in AWS before reusing this design outside the lab.

Implementation Summary

1. Created VPC A and VPC B with non-overlapping CIDR ranges.
2. Created the public subnet in VPC A and private subnets in VPC A and VPC B.
3. Created and attached the Internet Gateway to VPC A.
4. Created route tables and explicitly associated each subnet with the intended route table.
5. Created security groups and launched the bastion, App-A, and App-B EC2 instances.
6. Configured SSH access from the local workstation to the bastion and private instances through SSH `ProxyJump` entries.
7. Tested App-A to App-B connectivity before peering; the ping failed as expected.
8. Created and activated the VPC peering connection.
9. Added peering routes to the route tables in both VPCs and the public route table used by the bastion subnet.
10. Verified ping connectivity in both directions and SSH access to App-B.

11. Removed App-B's inbound ICMP rule temporarily to produce a controlled failure, restored the rule, and verified recovery.

Validation and Test Results

Test	Expected result	Observed result
App-A to App-B ping before peering	Fail because no peering route exists	**Passed** — 100% packet loss in baseline test
VPC peering status	Active	**Passed** — status shown as Active
App-A to App-B ping after routing	Succeed	**Passed** — 4 packets received, 0% loss
App-B to App-A ping after routing	Succeed	**Passed** — 4 packets received, 0% loss
SSH from workstation to bastion	Connect	**Passed**
SSH from workstation to App-A via bastion	Connect	**Passed**
SSH from workstation to App-B via bastion	Connect	**Passed**
Controlled ICMP failure after removing App-B ICMP rule	Ping fails	**Passed** — 100% packet loss while the rule was absent
Recovery after restoring App-B ICMP rule	Ping succeeds again	**Passed**

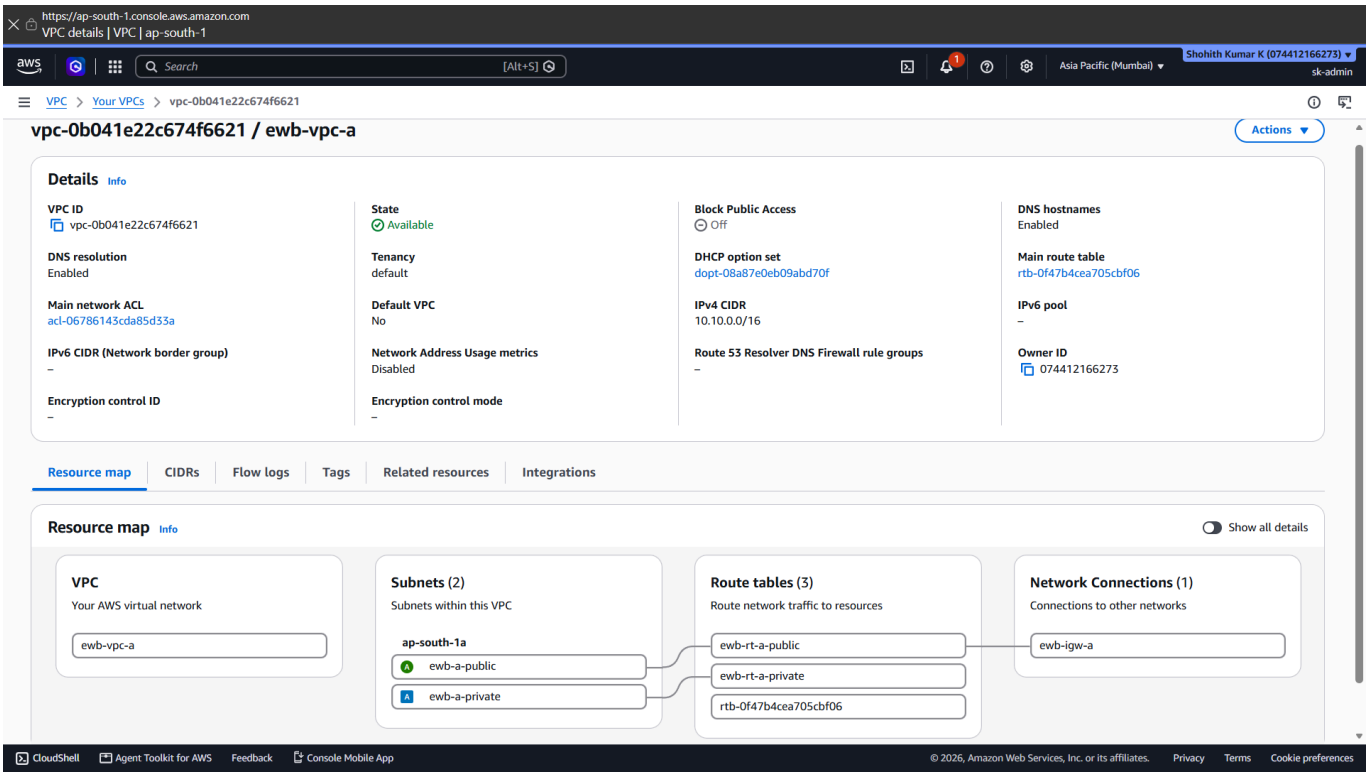
The results demonstrate that peering alone is not sufficient: the connection must be active, appropriate routes must exist on both sides, and security rules must allow the intended traffic.

Screenshots and Evidence

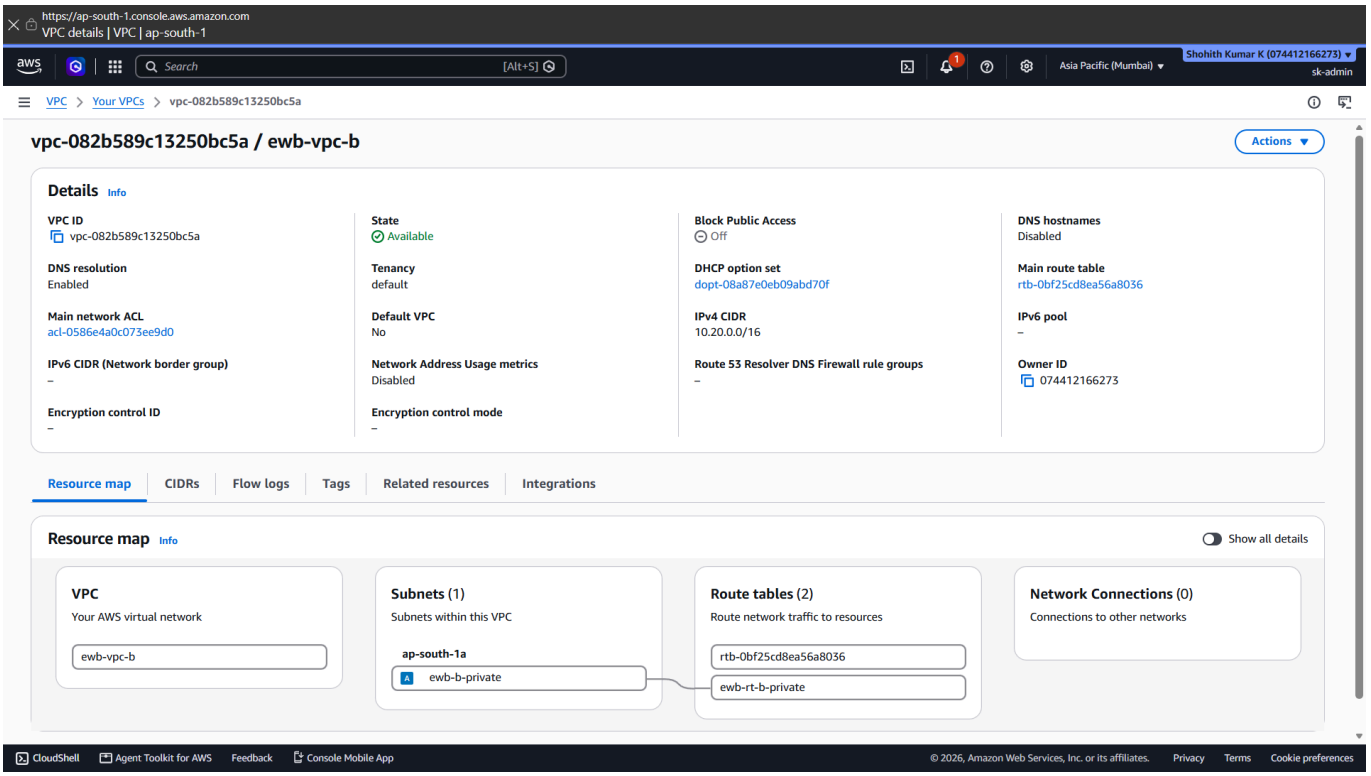
The following screenshots are included in the `screenshots/` folder. They are embedded below so GitHub can display the evidence directly in this README. Click an image to open it at full size.

1. VPCs and subnets

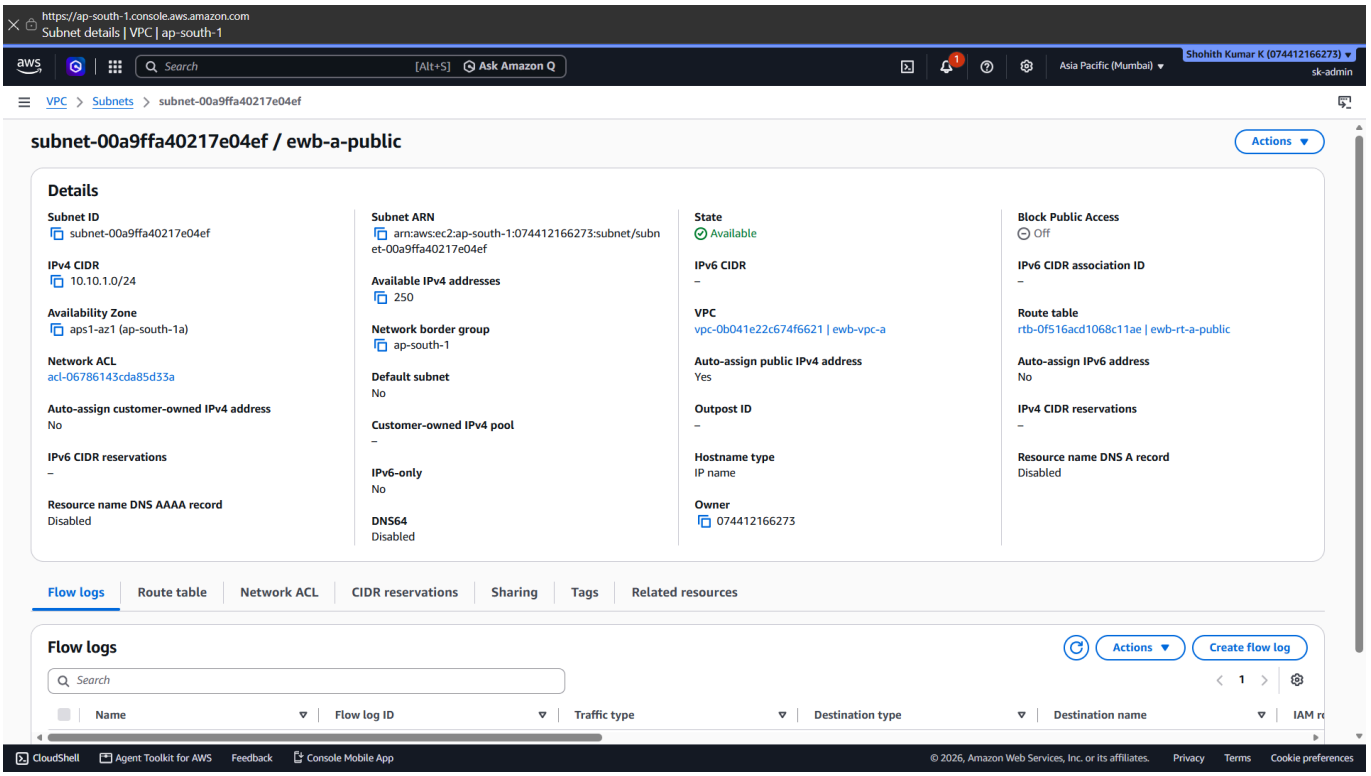
VPC A details



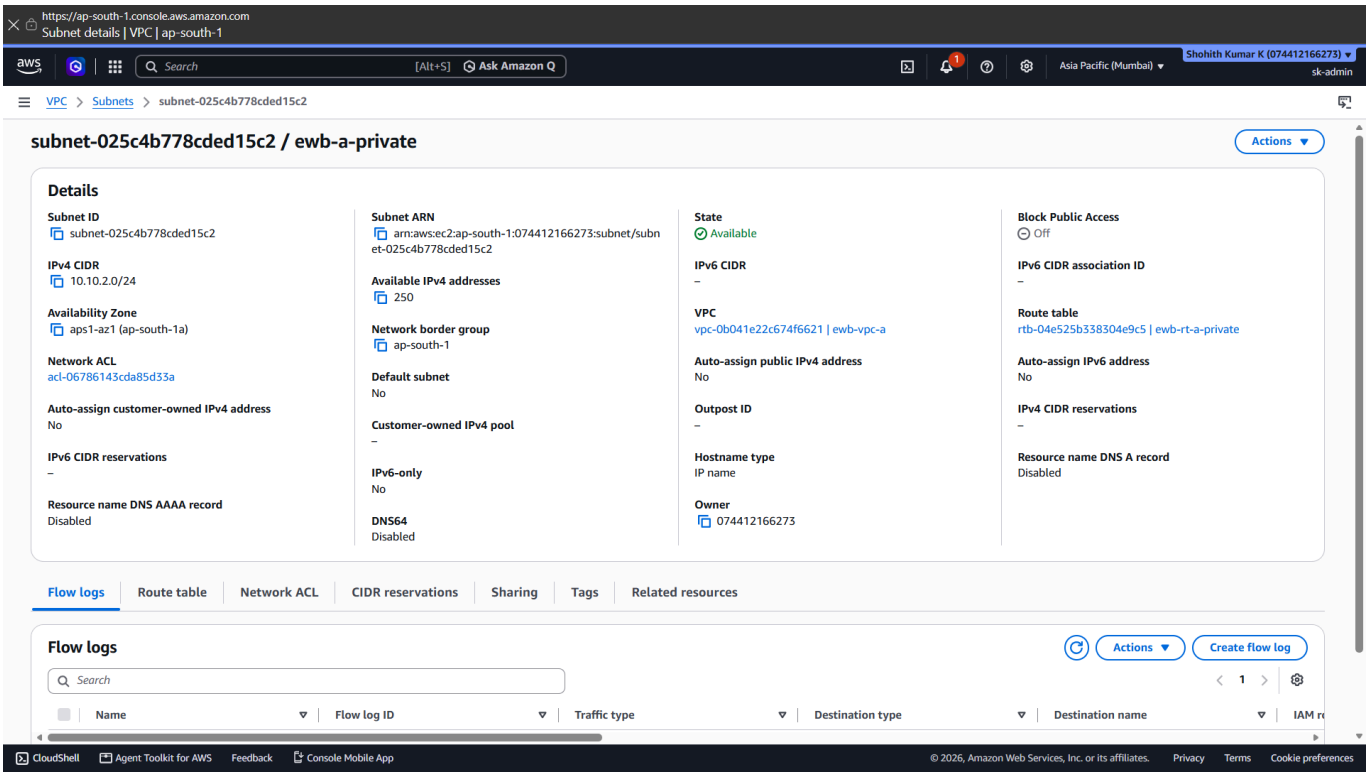
VPC B details



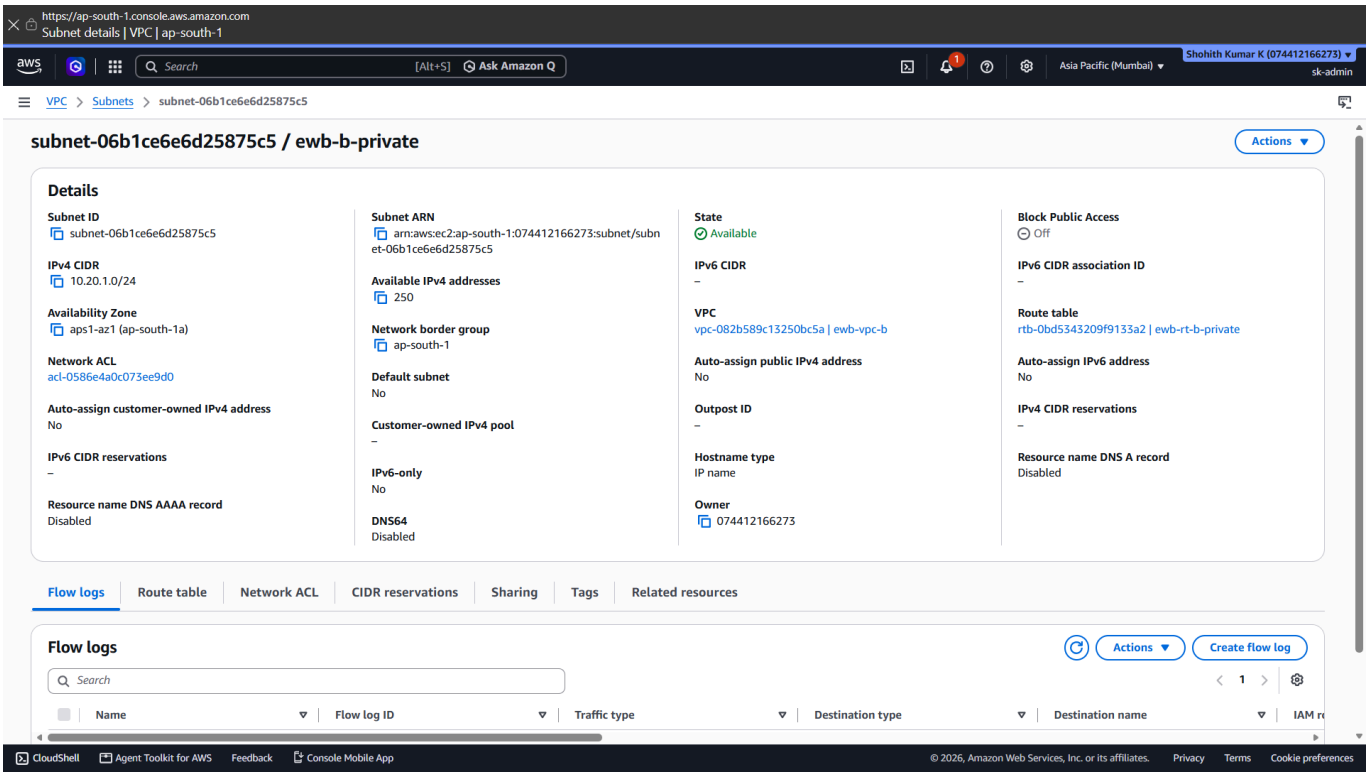
Public subnet in VPC A



Private subnet in VPC A

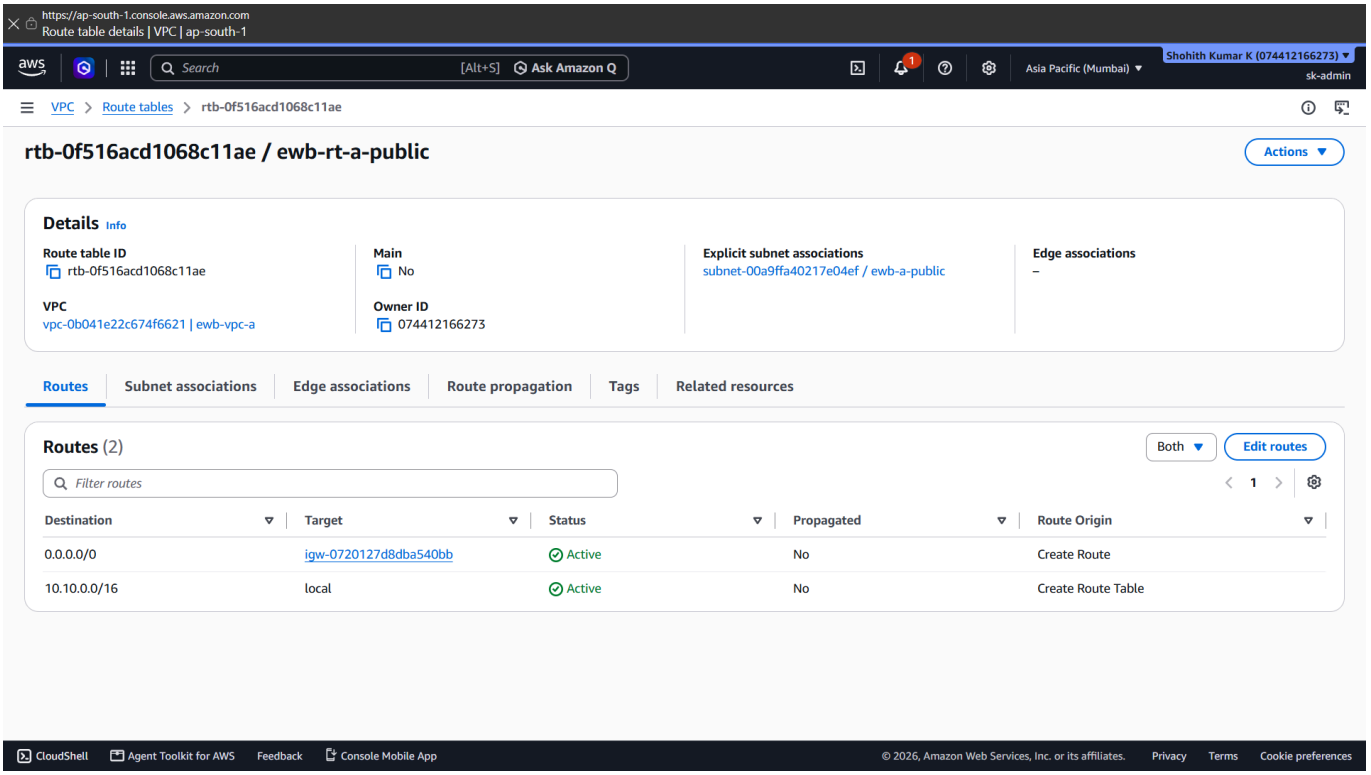


Private subnet in VPC B

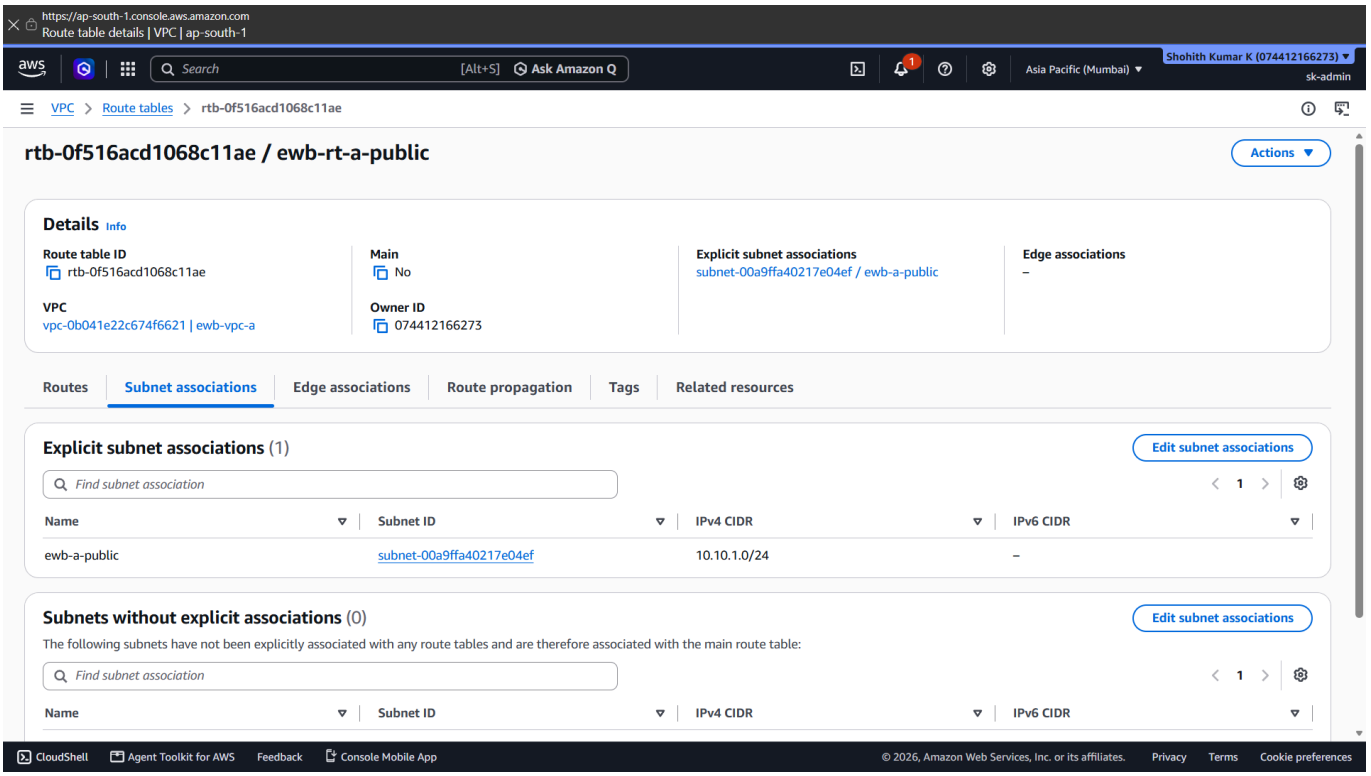


2. Initial route tables and subnet associations

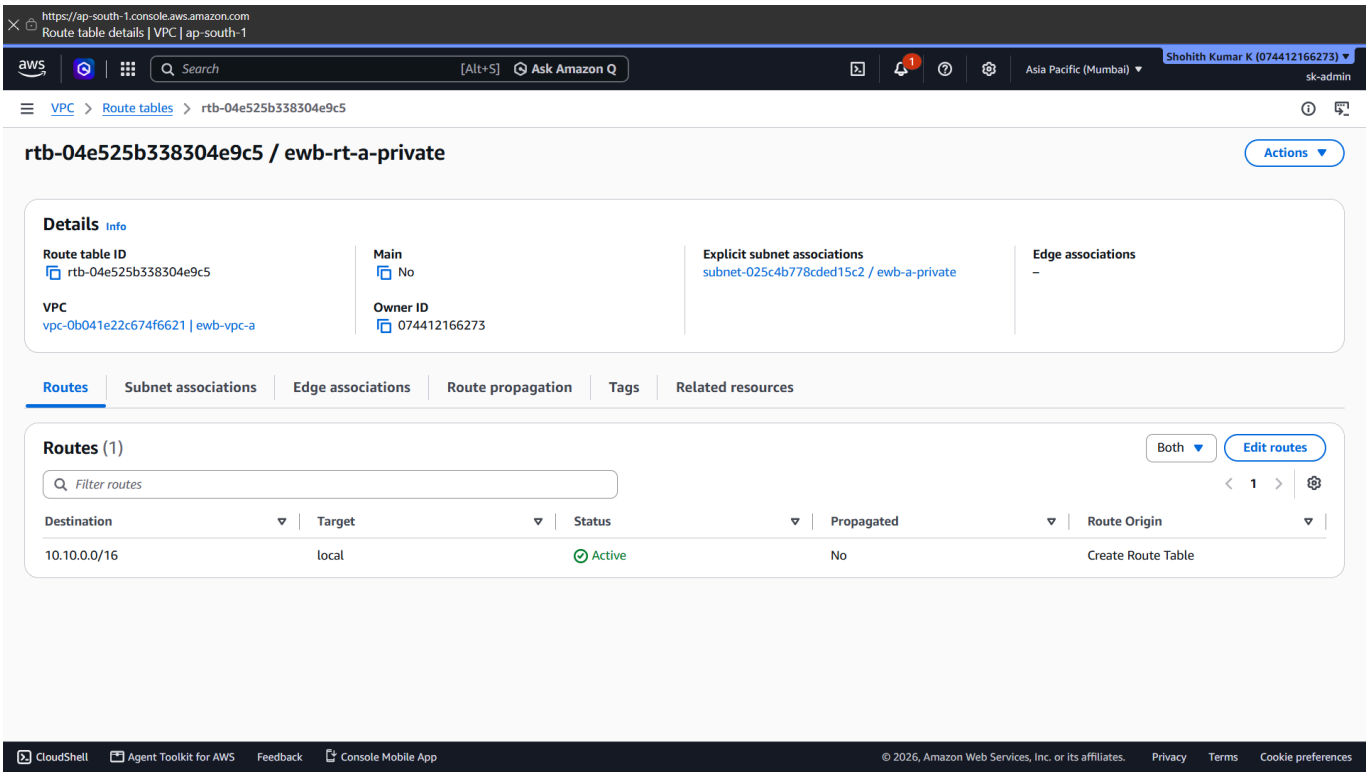
Public route table A



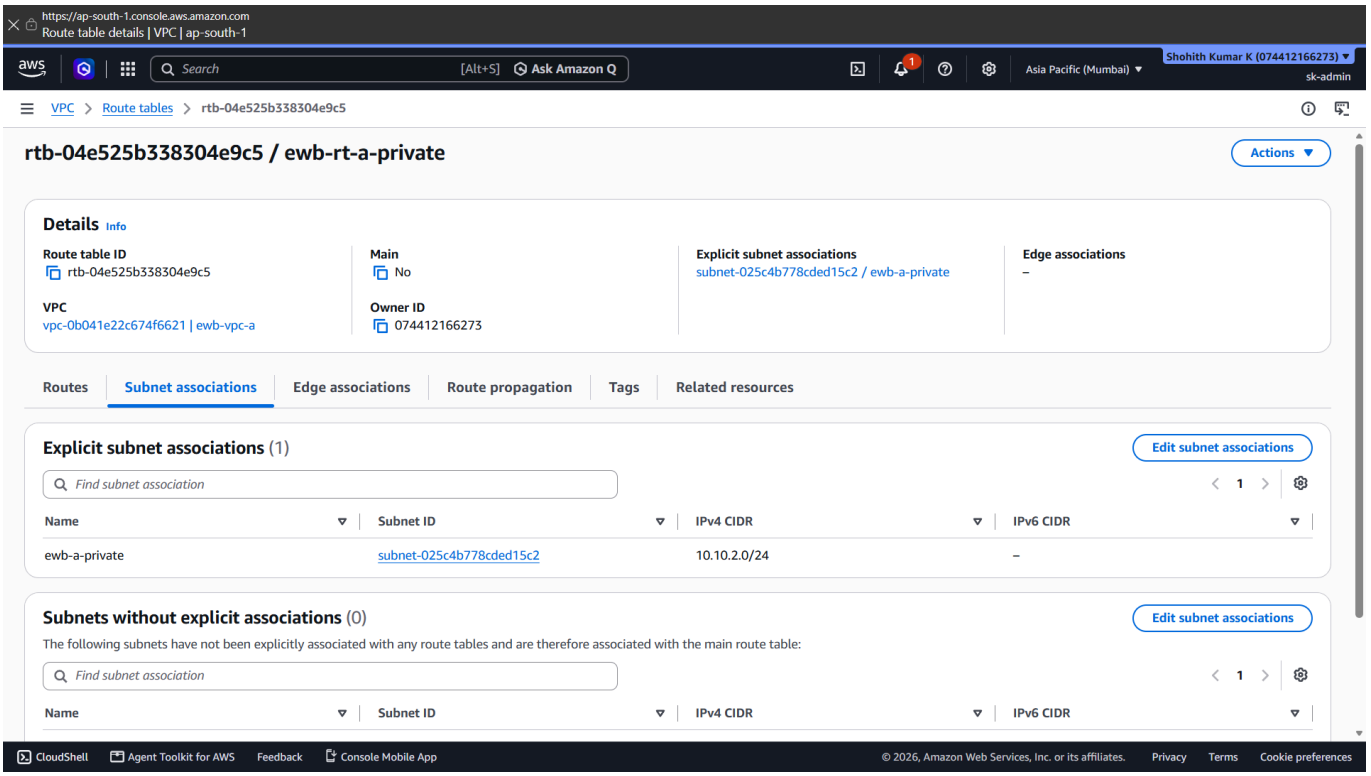
Public subnet association



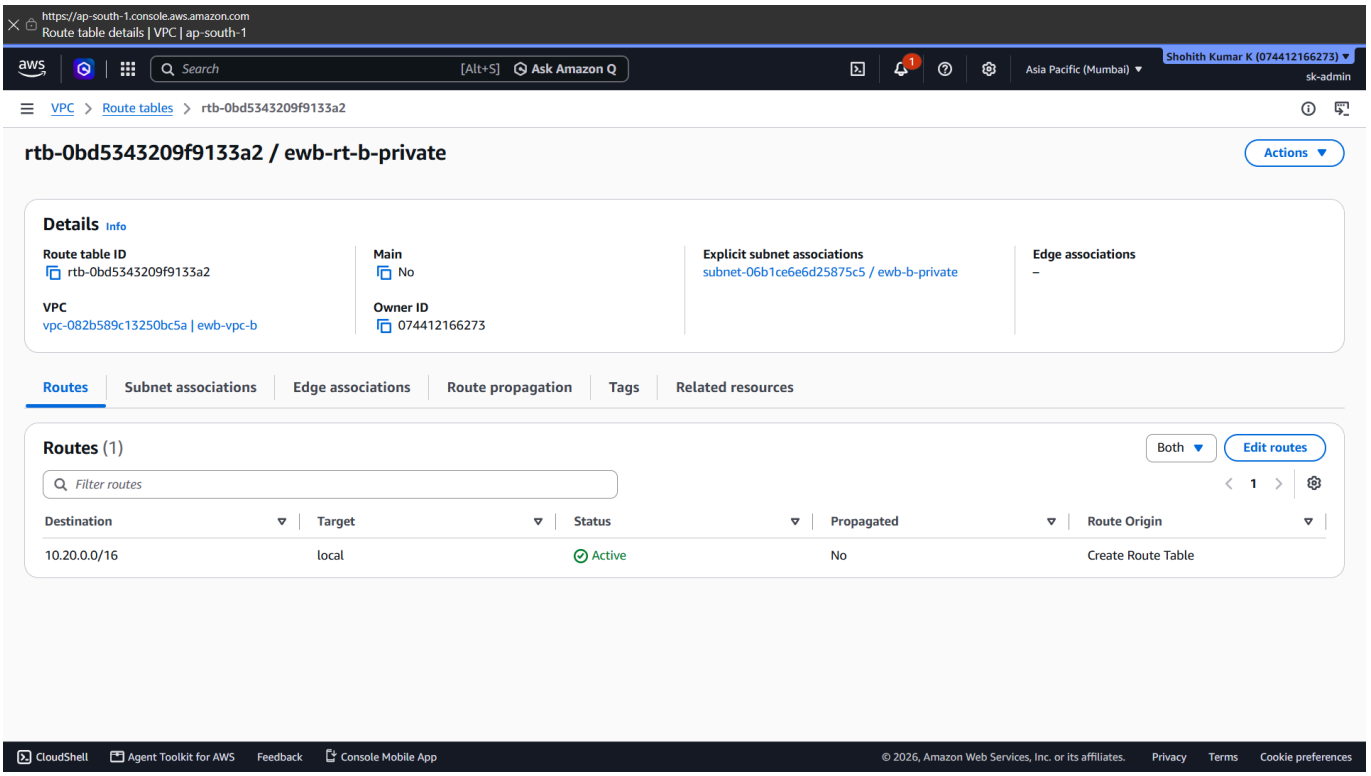
Private route table A



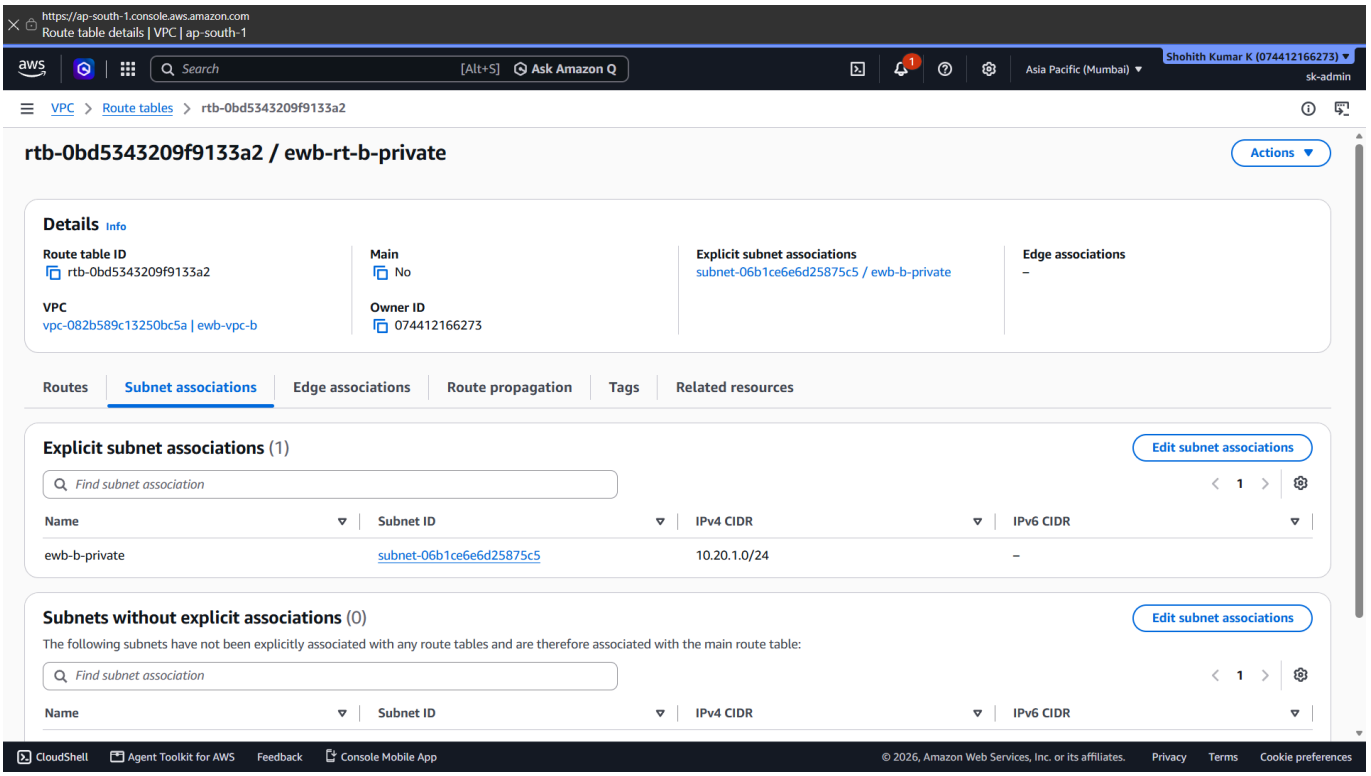
Private subnet A association



Private route table B

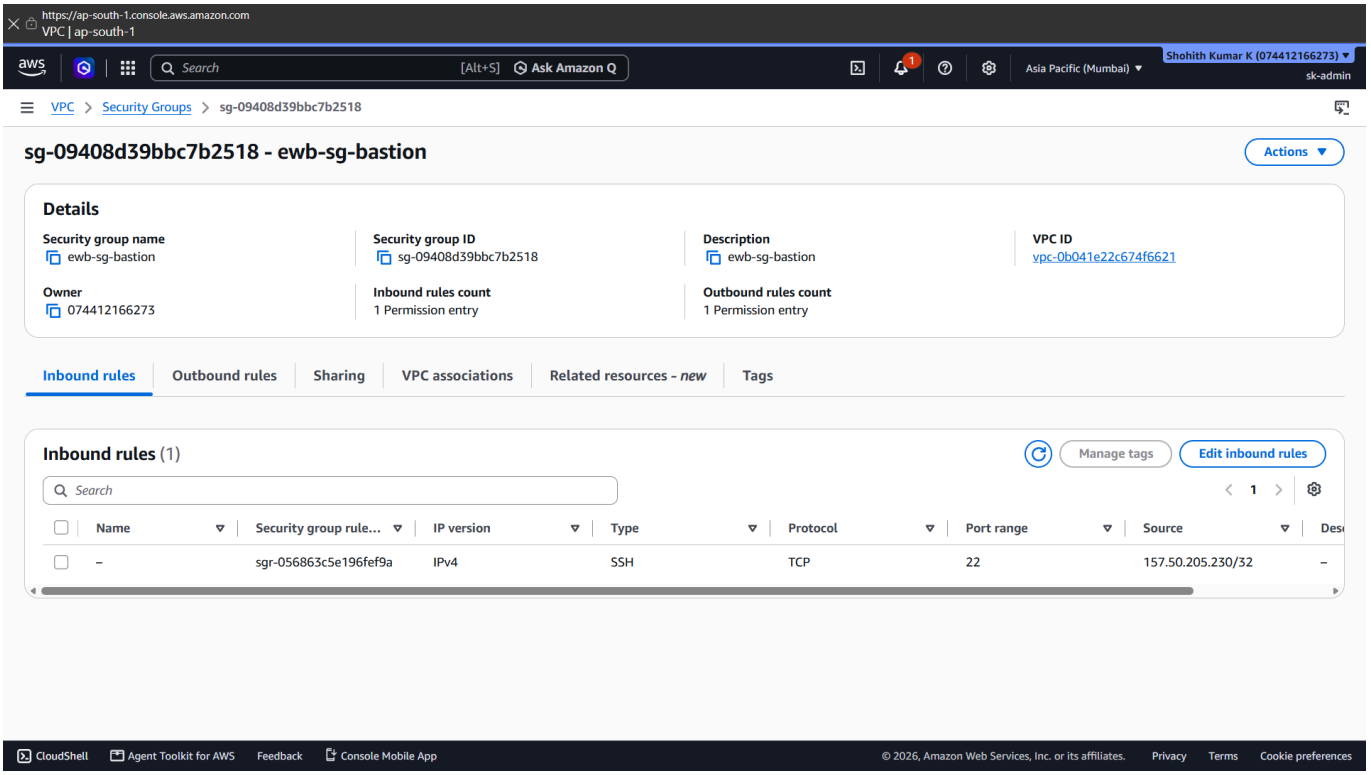


Private subnet B association

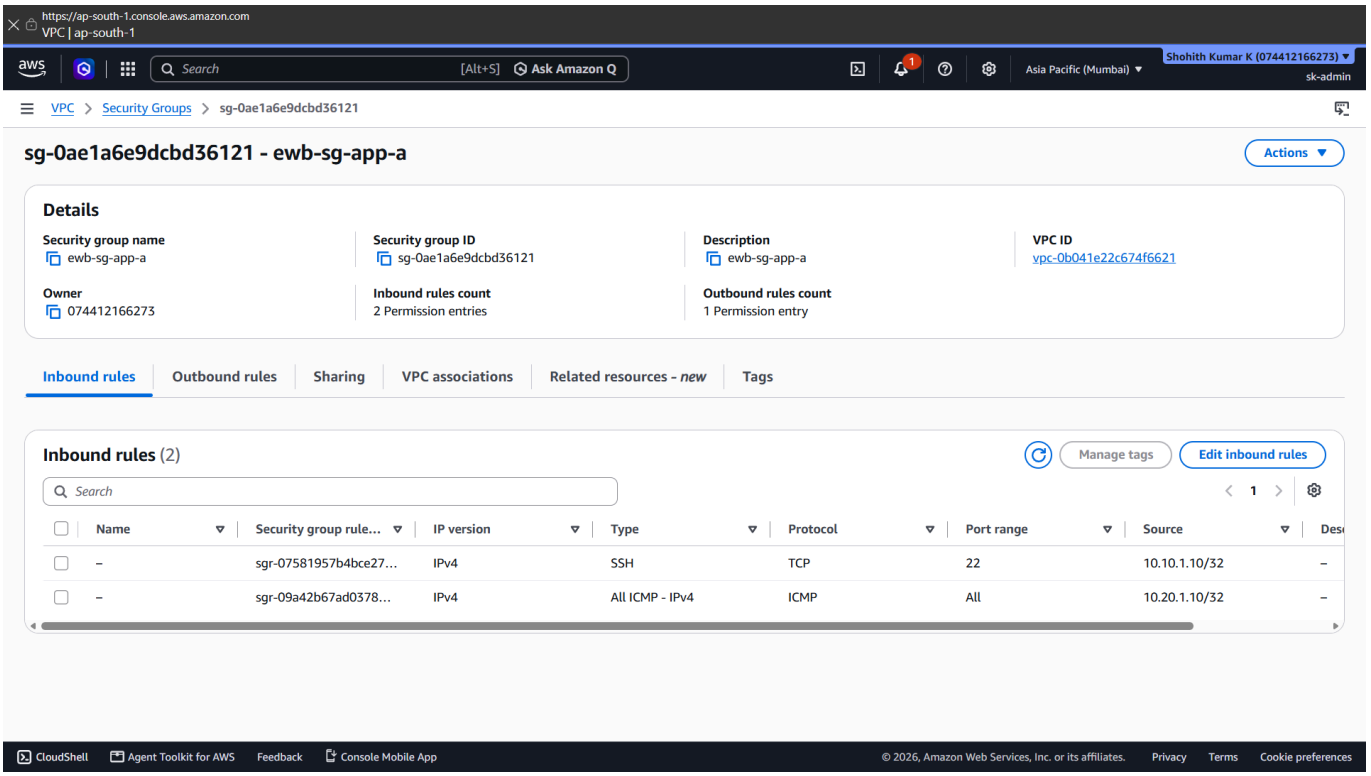


3. Security groups and EC2 instances

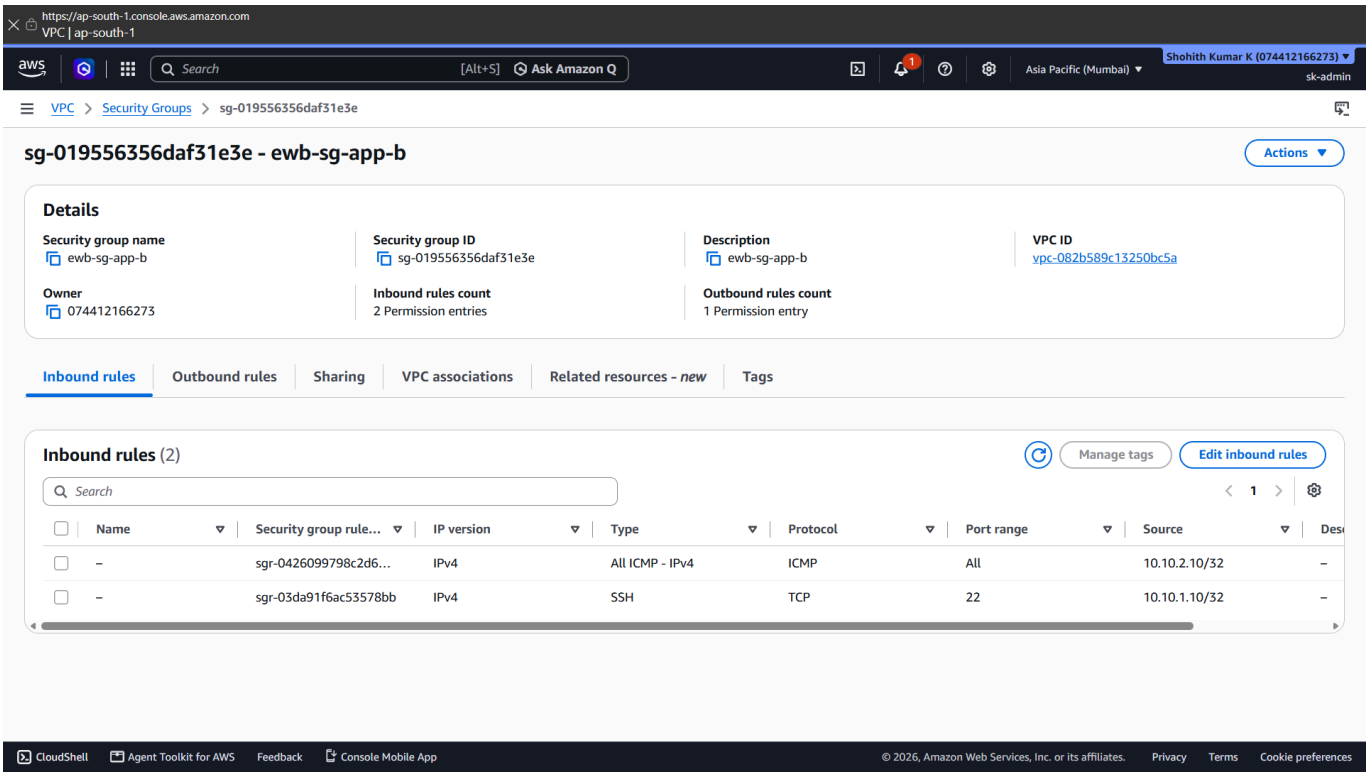
Bastion security group inbound rules



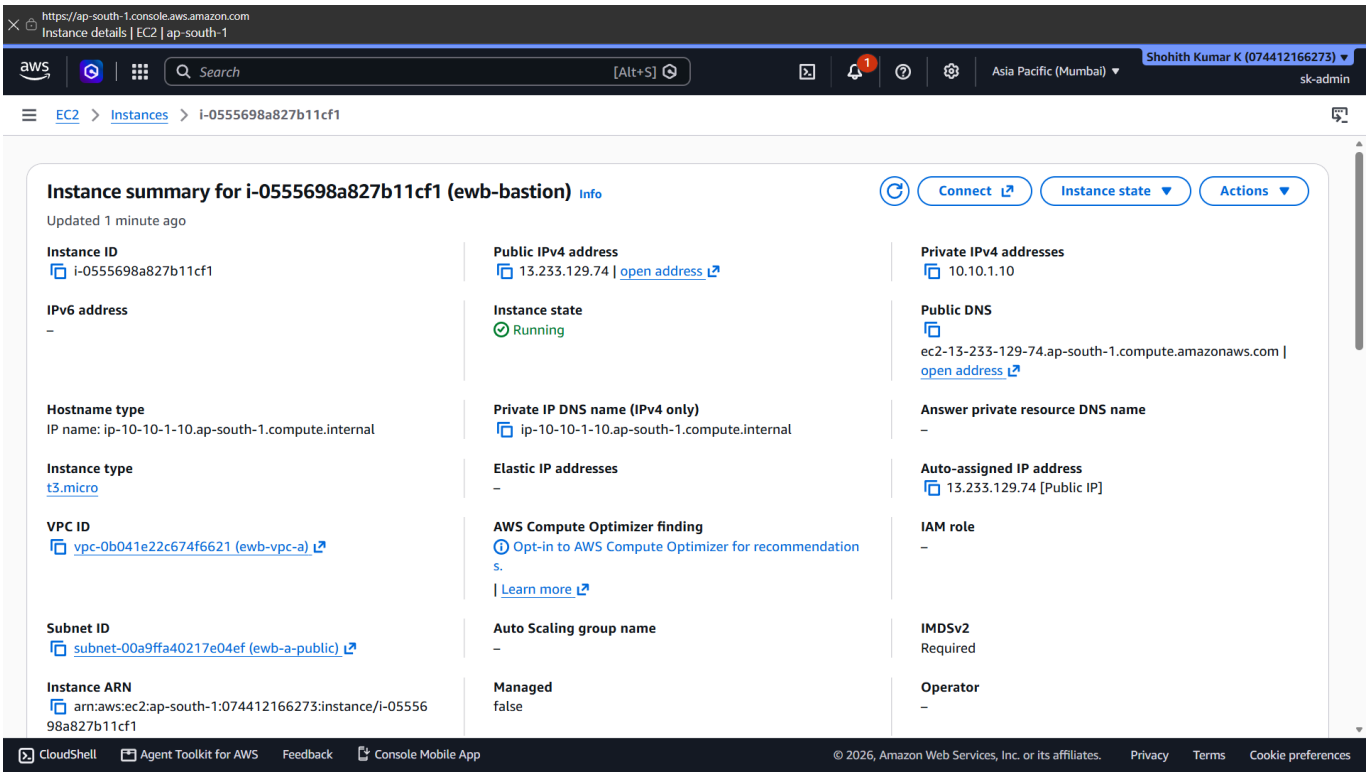
App-A security group inbound rules



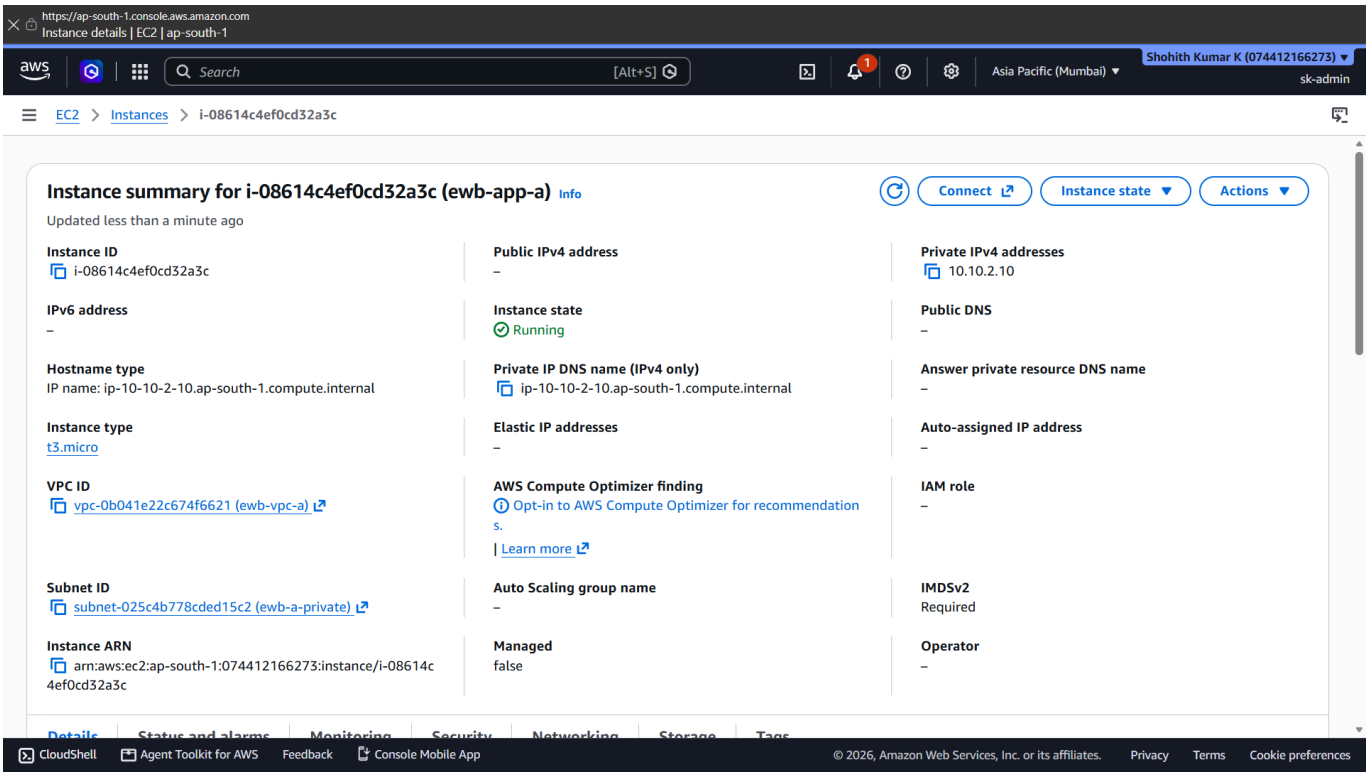
App-B security group inbound rules



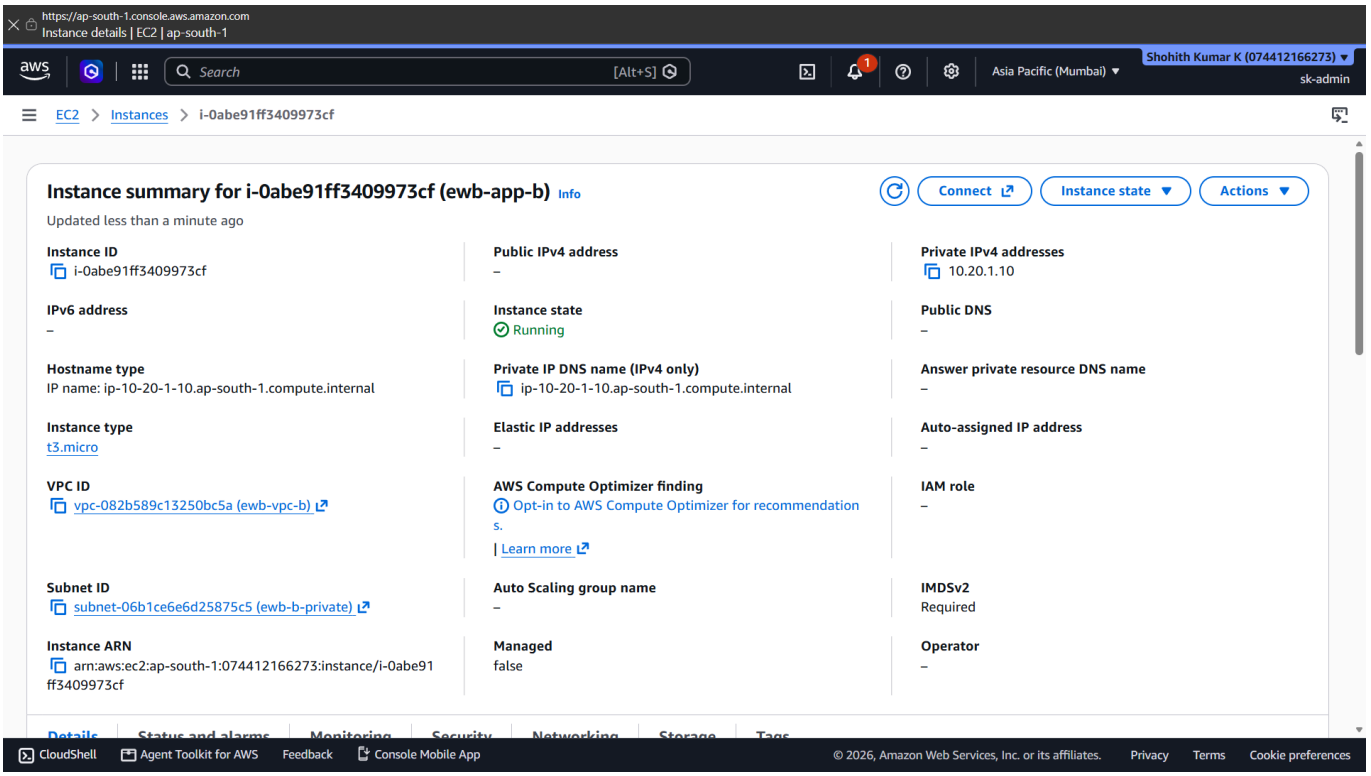
Bastion EC2 details



App-A EC2 details

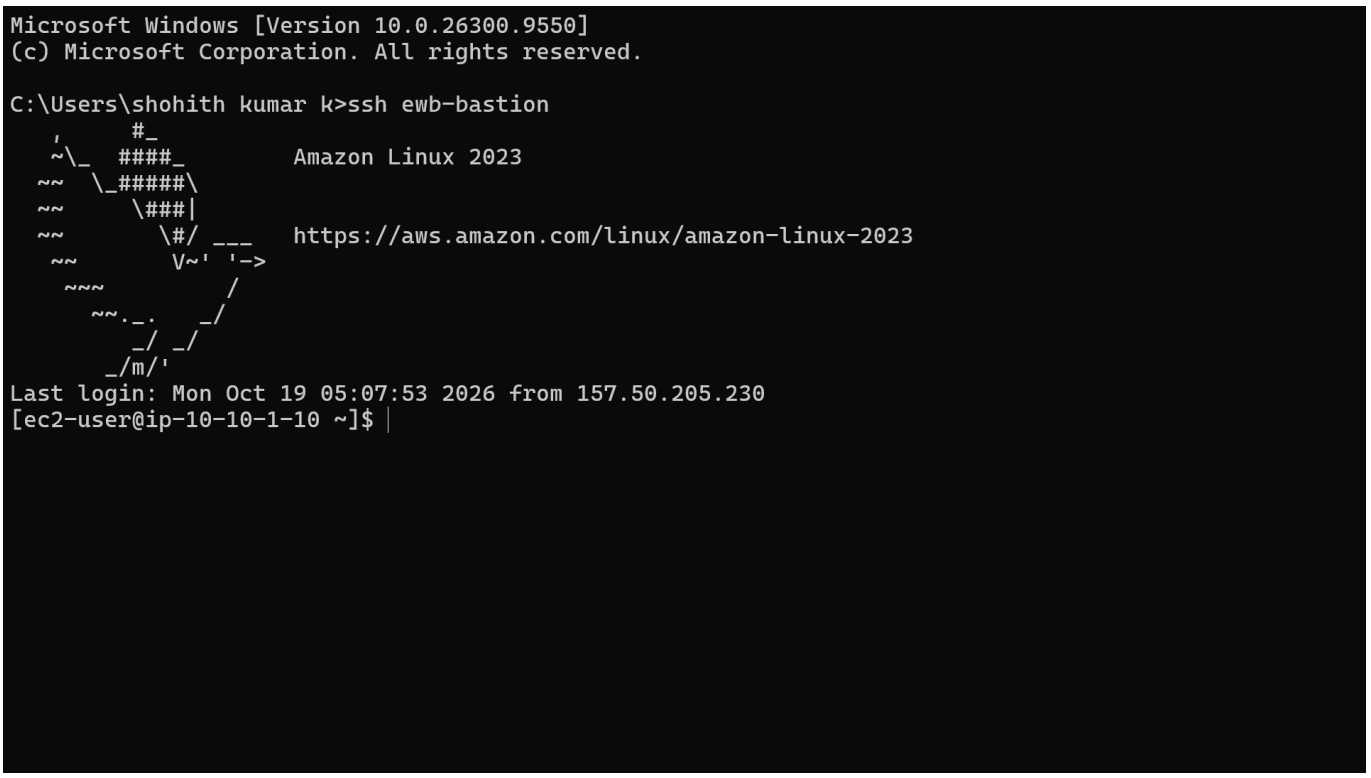


App-B EC2 details



4. Initial access and baseline test

SSH connection to the bastion host



SSH connection to App-A through the bastion

```
Microsoft Windows [Version 10.0.26300.9550]
(c) Microsoft Corporation. All rights reserved.

C:\Users\shohith kumar k>ssh ewb-app-a
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Mon Oct 19 04:39:05 2026 from 10.10.1.10
[ec2-user@ip-10-10-2-10 ~]$
```

Baseline ping failure before VPC peering

```
Microsoft Windows [Version 10.0.26300.9550]
(c) Microsoft Corporation. All rights reserved.

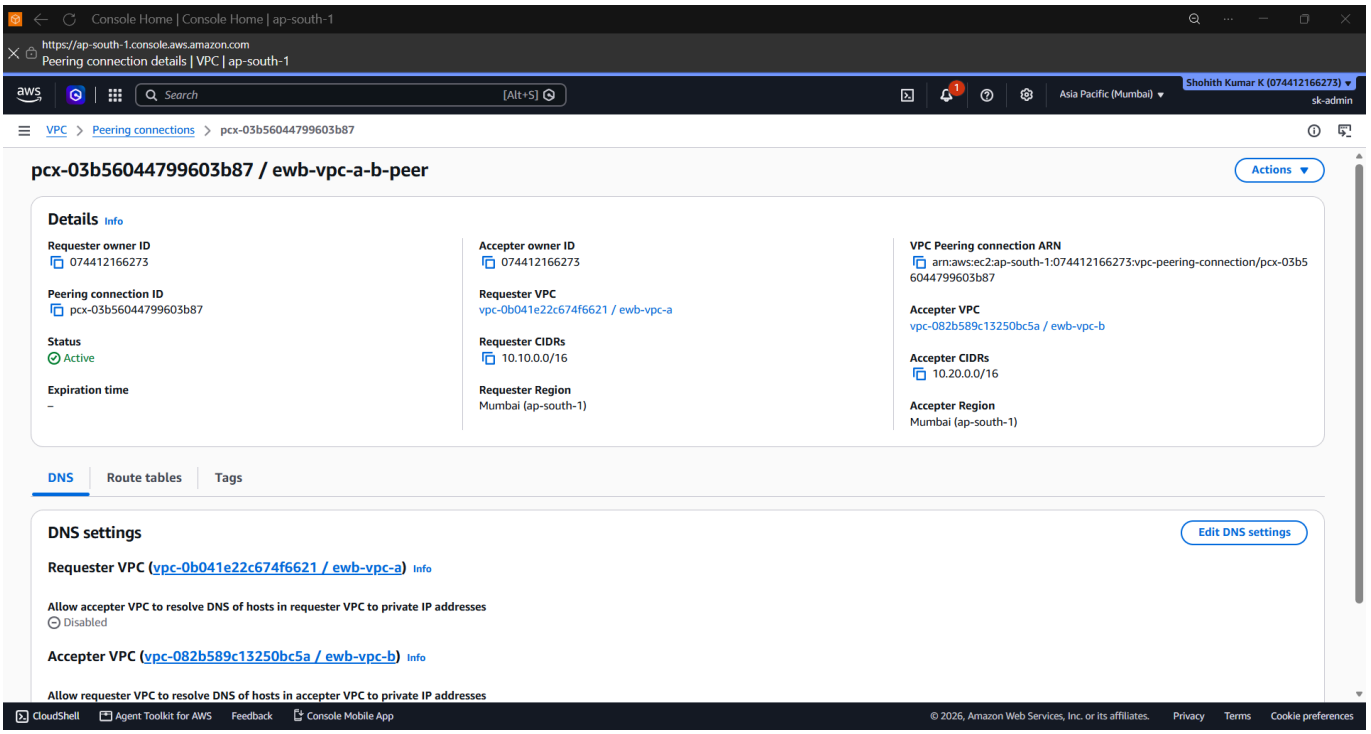
C:\Users\shohith kumar k>ssh ewb-app-a
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Mon Oct 19 05:06:57 2026 from 10.10.1.10
[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3088ms

[ec2-user@ip-10-10-2-10 ~]$
```

5. Peering and connectivity validation

VPC peering connection — Active



Successful ping: App-A to App-B

```
Microsoft Windows [Version 10.0.26300.9550]
(c) Microsoft Corporation. All rights reserved.

C:\Users\shohith kumar k>ssh ewb-app-a
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Mon Oct 19 06:55:40 2026 from 10.10.1.10
[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.
64 bytes from 10.20.1.10: icmp_seq=1 ttl=127 time=0.194 ms
64 bytes from 10.20.1.10: icmp_seq=2 ttl=127 time=0.209 ms
64 bytes from 10.20.1.10: icmp_seq=3 ttl=127 time=0.275 ms
64 bytes from 10.20.1.10: icmp_seq=4 ttl=127 time=0.204 ms

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3138ms
rtt min/avg/max/mdev = 0.194/0.220/0.275/0.031 ms
[ec2-user@ip-10-10-2-10 ~]$
```

Successful ping: App-B to App-A

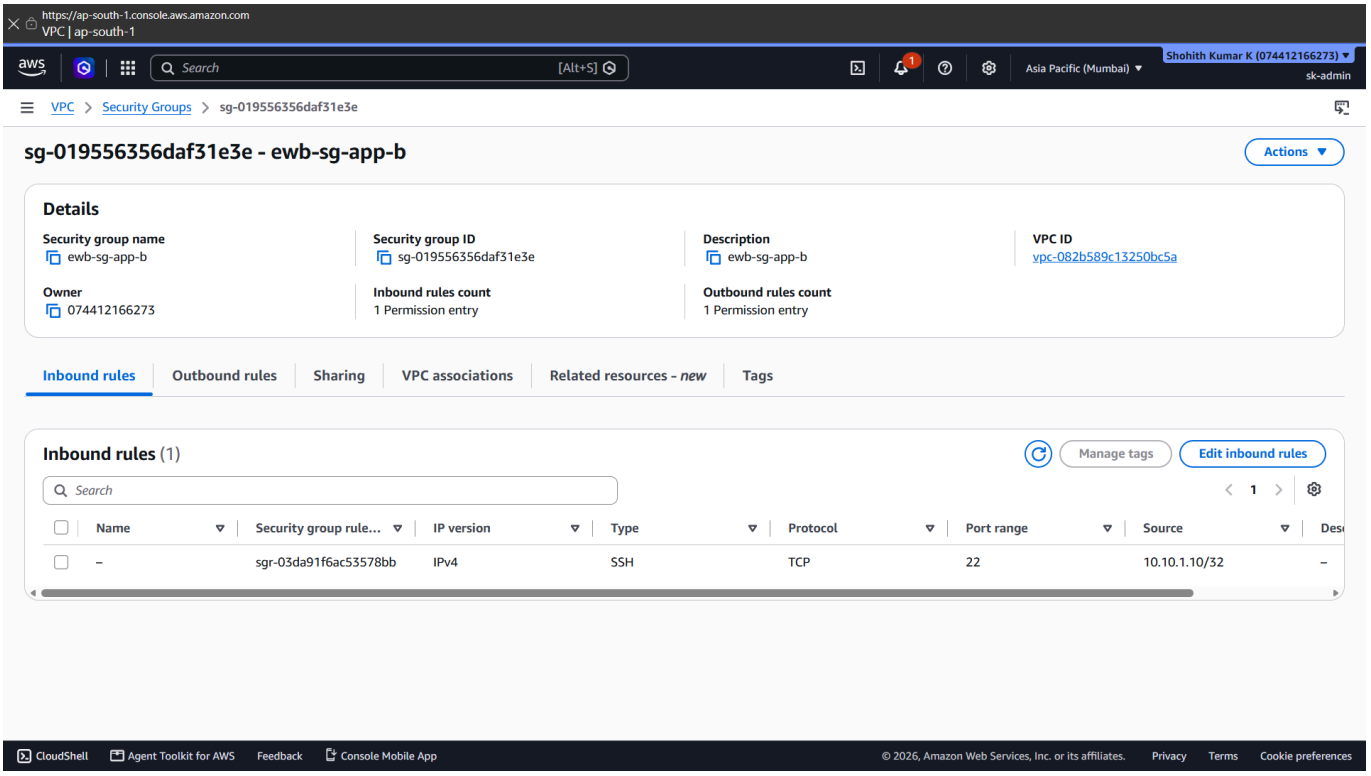
```
[ec2-user@ip-10-10-2-10 ~]$ exit
logout
Connection to 10.10.2.10 closed.

C:\Users\shohith kumar k>ssh ewb-app-b
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Mon Oct 19 06:55:49 2026 from 10.10.1.10
[ec2-user@ip-10-20-1-10 ~]$ ping -c 4 10.10.2.10
PING 10.10.2.10 (10.10.2.10) 56(84) bytes of data.
64 bytes from 10.10.2.10: icmp_seq=1 ttl=127 time=0.203 ms
64 bytes from 10.10.2.10: icmp_seq=2 ttl=127 time=0.239 ms
64 bytes from 10.10.2.10: icmp_seq=3 ttl=127 time=0.287 ms
64 bytes from 10.10.2.10: icmp_seq=4 ttl=127 time=0.209 ms

--- 10.10.2.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3097ms
rtt min/avg/max/mdev = 0.203/0.234/0.287/0.033 ms
[ec2-user@ip-10-20-1-10 ~]$
```

6. Controlled failure and recovery

App-B security group during the controlled failure



App-A to App-B ping during the controlled failure

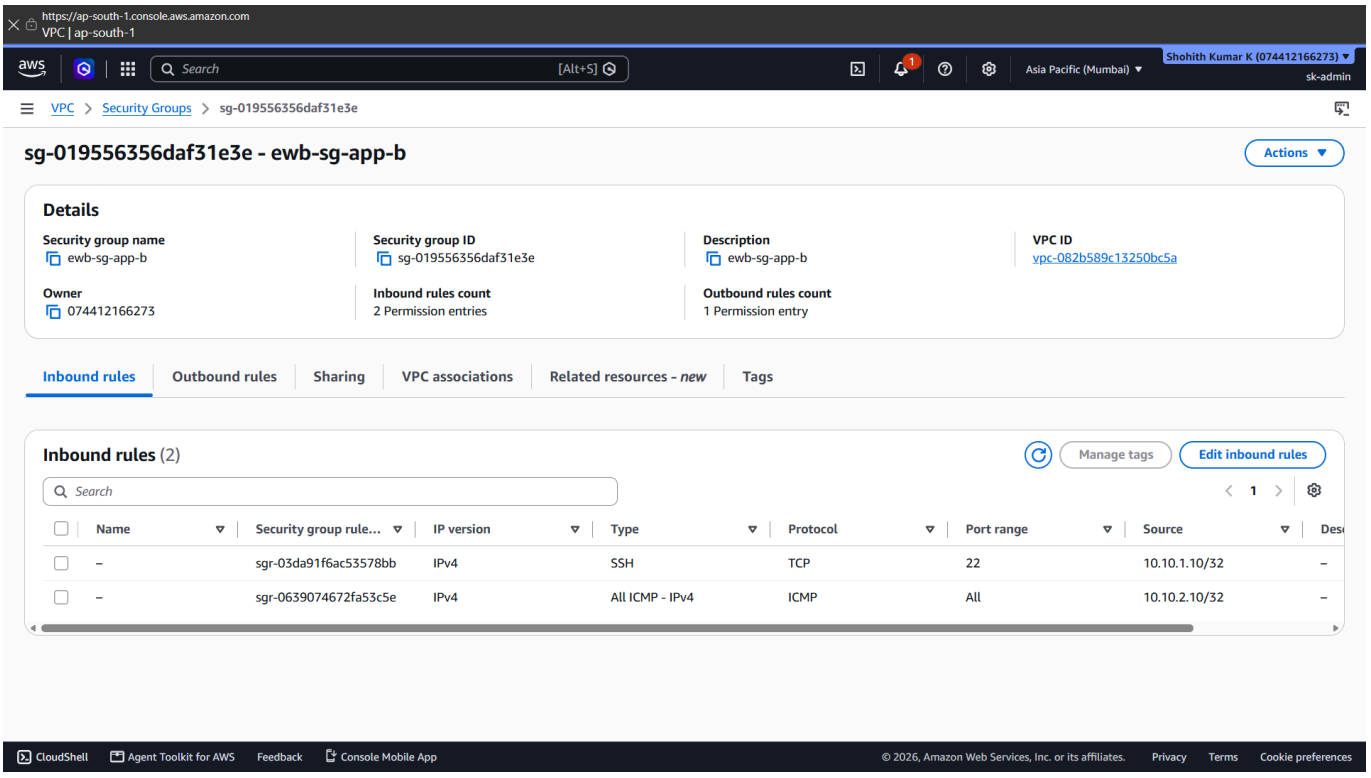
```
--- 10.10.2.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3097ms
rtt min/avg/max/mdev = 0.203/0.234/0.287/0.033 ms
[ec2-user@ip-10-20-1-10 ~]$ exit
logout
Connection to 10.20.1.10 closed.

C:\Users\shohith kumar k>ssh ewb-app-a
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Mon Oct 19 07:01:42 2026 from 10.10.1.10
[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3128ms

[ec2-user@ip-10-10-2-10 ~]$
```

App-B ICMP rule restored



Ping successful after recovery

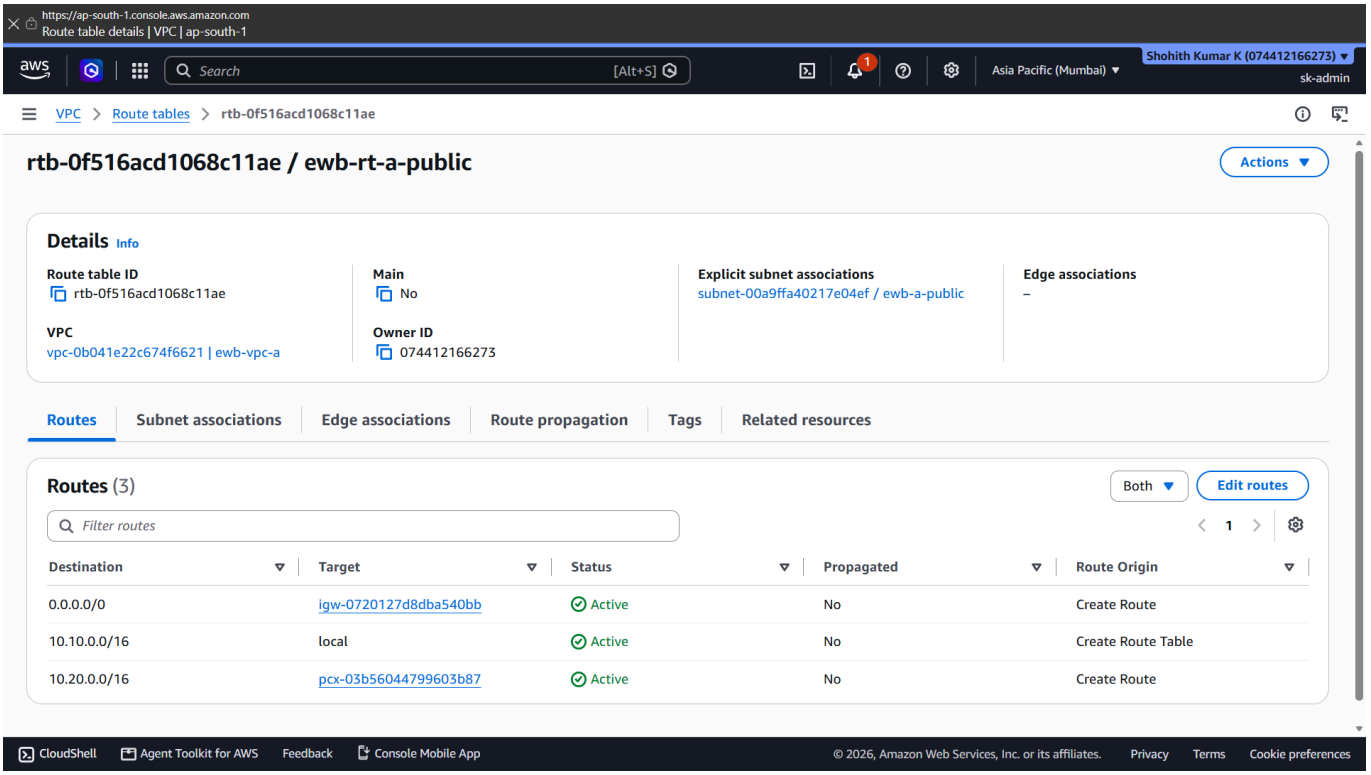
```
rtt min/avg/max/mdev = 0.197/0.208/0.217/0.007 ms
[ec2-user@ip-10-20-1-10 ~]$ client_loop: send disconnect: Connection reset
client_loop: send disconnect: Unknown error

C:\Users\shohith kumar k>ssh ewb-app-a
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Mon Oct 19 07:04:31 2026 from 10.10.1.10
[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.
64 bytes from 10.20.1.10: icmp_seq=1 ttl=127 time=0.205 ms
64 bytes from 10.20.1.10: icmp_seq=2 ttl=127 time=0.214 ms
64 bytes from 10.20.1.10: icmp_seq=3 ttl=127 time=0.212 ms
64 bytes from 10.20.1.10: icmp_seq=4 ttl=127 time=0.221 ms

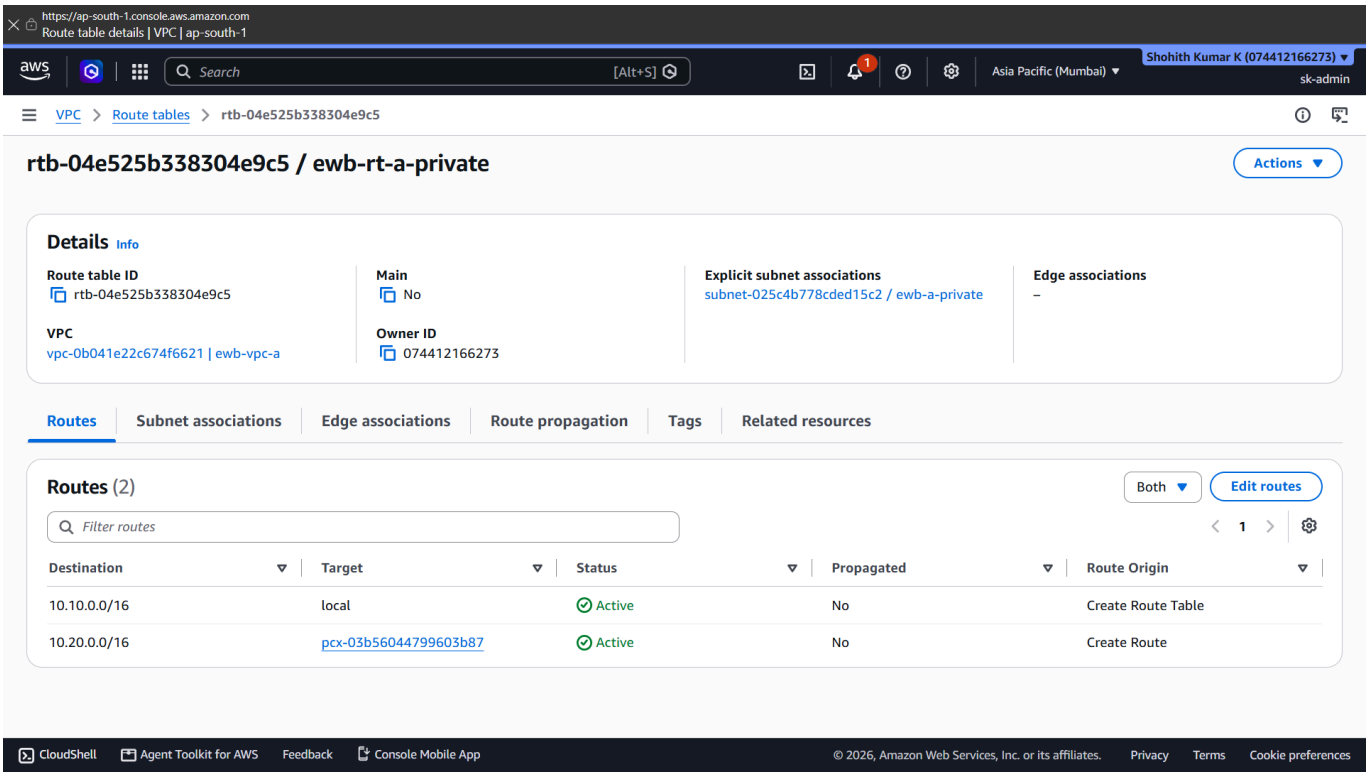
--- 10.20.1.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3128ms
rtt min/avg/max/mdev = 0.205/0.213/0.221/0.005 ms
[ec2-user@ip-10-10-2-10 ~]$
```

7. Final route tables and App-B SSH

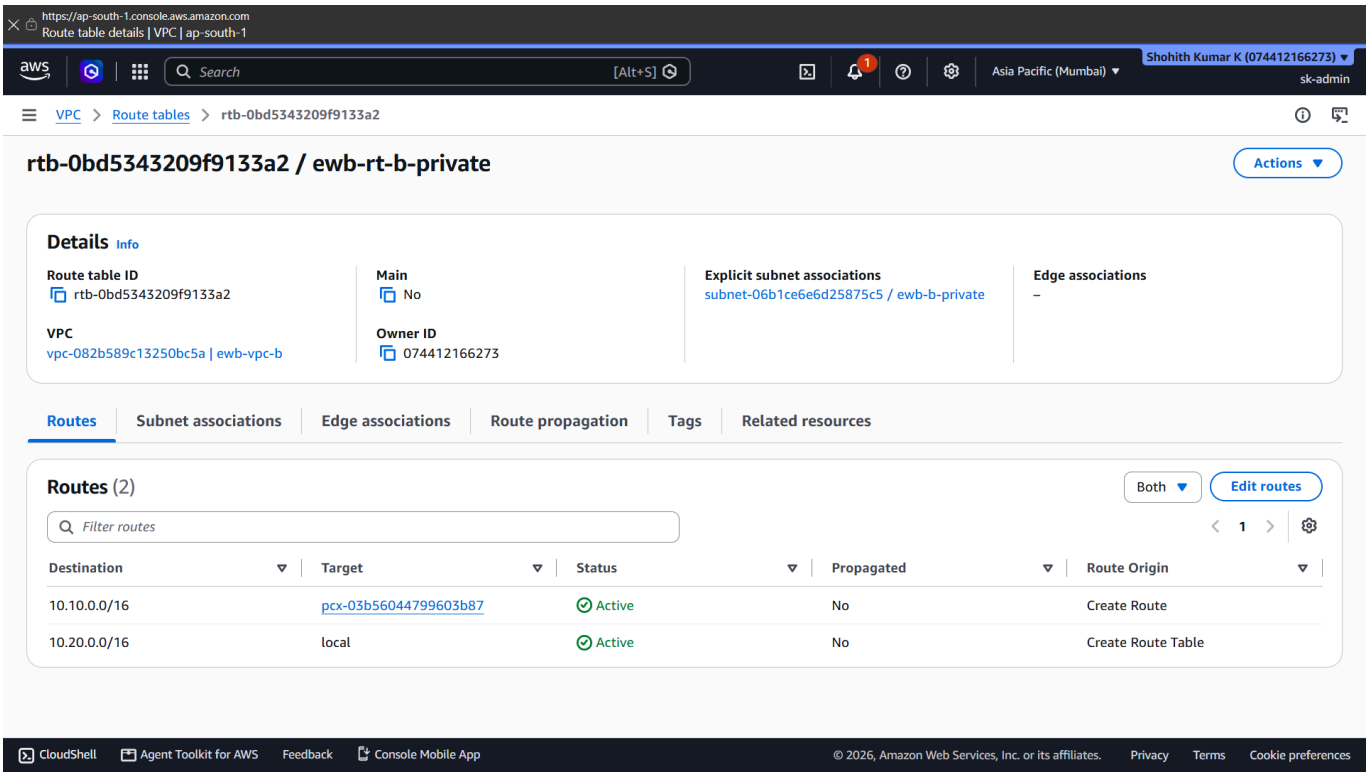
Final public route table A



Private route table A with peering route



Private route table B with peering route



SSH connection to App-B through the bastion

```
Microsoft Windows [Version 10.0.26300.9550]
(c) Microsoft Corporation. All rights reserved.

C:\Users\shohith kumar k>ssh ewb-app-b
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Mon Oct 19 07:08:00 2026 from 10.10.1.10
[ec2-user@ip-10-20-1-10 ~]$
```

Key Learnings

- VPC peering enables private connectivity between two VPCs with non-overlapping CIDR blocks.
- Peering connections do not automatically add routes; route tables must be configured on both sides.
- The route table associated with the bastion's subnet also needs a route to the destination VPC for the bastion to reach private instances across the peering connection.
- Security groups control which traffic is allowed to reach an instance, even when routing is correct.
- A baseline test, a controlled failure, and a recovery test help demonstrate that the configuration behaves as expected.
- SSH `ProxyJump` makes it possible to connect to private EC2 instances without assigning them public IP addresses.

Cleanup

After completing the VPC Peering lab and capturing the required screenshots, all lab resources were successfully cleaned up to prevent ongoing AWS charges.

Cleanup Tasks Completed

- [✓] Terminated the three EC2 instances: `ewb-bastion`, `ewb-app-a`, and `ewb-app-b`.
- [✓] Deleted the VPC peering connection.
- [✓] Confirmed that no NAT Gateway was required for the documented design.
- [✓] Detached and deleted the Internet Gateway.
- [✓] Deleted the custom route tables and subnets.
- [✓] Deleted both VPCs after removing their dependent resources.
- [✓] Reviewed Elastic IP addresses, snapshots, and other potentially billable resources.
- [✓] Reviewed the AWS billing console for unwanted remaining resources.

Cleanup Status

Status: Completed

All lab resources were cleaned up after successfully validating:

- VPC peering connectivity between VPC A and VPC B.
- Bidirectional ICMP connectivity between App-A and App-B.
- SSH access to private instances through the bastion host.
- Controlled network failure and successful recovery.
